LazyModuleMixin#

https://pytorch.org/docs/stable/generated/torch.nn.modules.lazy.LazyModuleMi

Description:

A mixin for modules that lazily initialize parameters, also known as “lazy modules”. Modules that lazily initialize parameters, or “lazy modules”, derive the shapes of their parameters from the first input(s) to their forward method. Until that first forward they contain torch.nn.UninitializedParameter s that should not be accessed or used, and afterward they contain regular torch.nn.Parameter s. Lazy modules are convenient since they don’t require computing some module arguments, like the in_features argument of a typical torch.nn.Linear. After construction, networks with lazy modules should first be converted to the desired dtype and placed on the expected device. This is because lazy modules only perform shape inference so the usual dtype and device placement behavior applies. The lazy modules should then perform “dry runs” to initialize all the components in the module. These “dry runs” send inputs of the correct size, dtype, and device through the network and to each one of its lazy modules. After this the network can be used as usual. A final caveat when using lazy modules is that the order of initialization of a network’s parameters may change, since the lazy modules are

Function Signature

```
class torch.nn.modules.lazy.LazyModuleMixin(*args, **kwargs)
[source]#

has_uninitialized_params()[source]#

initialize_parameters(*args, **kwargs)[source]#
```

Code Examples

```
>>> class LazyMLP(torch.nn.Module):
...     def __init__(self) -> None:
...         super().__init__()
...         self.fc1 = torch.nn.LazyLinear(10)
...         self.relu1 = torch.nn.ReLU()
...         self.fc2 = torch.nn.LazyLinear(1)
...         self.relu2 = torch.nn.ReLU()
...
...     def forward(self, input):
...         x = self.relu1(self.fc1(input))
...         y = self.relu2(self.fc2(x))
...         return y
>>> # constructs a network with lazy modules
>>> lazy_mlp = LazyMLP()
>>> # transforms the network's device and dtype
>>> # NOTE: these transforms can and should be applied after
construction and before any 'dry runs'
>>> lazy_mlp = lazy_mlp.cuda()
>>> lazy_mlp
LazyMLP( (fc1): LazyLinear(in_features=0, out_features=10,
```

```
bias=True)
  (relu1): ReLU()
  (fc2): LazyLinear(in_features=0, out_features=1, bias=True)
  (relu2): ReLU()
)
>>> # performs a dry run to initialize the network's lazy
modules
>>> lazy_mlp(torch.ones(10, 10).cuda())
>>> # after initialization, LazyLinear modules become regular
Linear modules
>>> lazy_mlp
LazyMLP(
  (fc1): Linear(in_features=10, out_features=10, bias=True)
  (relu1): ReLU()
  (fc2): Linear(in_features=10, out_features=1, bias=True)
  (relu2): ReLU()
)
>>> # attaches an optimizer, since parameters can now be used as
usual
>>> optim = torch.optim.SGD(lazy_mlp.parameters(), lr=0.01)
```

```
>>> lazy_mlp = LazyMLP()
>>> # The state dict shows the uninitialized parameters
>>> lazy_mlp.state_dict()
OrderedDict({'fc1.weight': <UninitializedParameter>,
             'fc1.bias': <UninitializedParameter>,
             'fc2.weight': <UninitializedParameter>,
             'fc2.bias': <UninitializedParameter>})
```

```
>>> full_mlp = LazyMLP()
>>> # Dry run to initialize another module
>>> full_mlp.forward(torch.ones(10, 1))
>>> # Load an initialized state into a lazy module
>>> lazy_mlp.load_state_dict(full_mlp.state_dict())
>>> # The state dict now holds valid values
>>> lazy_mlp.state_dict()
OrderedDict([('fc1.weight',
              tensor([[-0.3837],
                      [ 0.0907],
                      [ 0.6708],
                      [-0.5223],
                      [-0.9028],
                      [ 0.2851],
                      [-0.4537],
                      [ 0.6813],
                      [ 0.5766],
                      [-0.8678]])),
             ('fc1.bias',
              tensor([-1.8832e+25,  4.5636e-41, -1.8832e+25,
4.5636e-41, -6.1598e-30,
                       4.5637e-41, -1.8788e+22,  4.5636e-41,
-2.0042e-31,  4.5637e-41])),
             ('fc2.weight',
              tensor([[ 0.1320,  0.2938,  0.0679,  0.2793,
0.1088, -0.1795, -0.2301,  0.2807,
                        0.2479,  0.1091]])),
             ('fc2.bias', tensor([0.0019]))])
```

Detailed Documentation

Documentation

A mixin for modules that lazily initialize parameters, also known as “lazy modules”.

Modules that lazily initialize parameters, or “lazy modules”, derive the shapes of their parameters from the first input(s) to their forward method. Until that first forward they contain torch.nn.UninitializedParameter s that should not be accessed or used, and afterward they contain regular torch.nn.Parameter s. Lazy modules are convenient since they don’t require computing some module arguments, like the in_features argument of a typical torch.nn.Linear.

After construction, networks with lazy modules should first be converted to the desired dtype and placed on the expected device. This is because lazy modules only perform shape inference so the usual dtype and device placement behavior applies. The lazy modules should then perform “dry runs” to initialize all the components in the module. These “dry runs” send inputs of the correct size, dtype, and device through the network and to each one of its lazy modules. After this the network can be used as usual.

A final caveat when using lazy modules is that the order of initialization of a network’s parameters may change, since the lazy modules are always initialized after other modules. For example, if the LazyMLP class defined above had a torch.nn.LazyLinear module first and then a regular torch.nn.Linear second, the second module would be initialized on construction and the first module would be initialized during the first dry run. This can cause the parameters of a network using lazy modules to be initialized differently than the parameters of a network without lazy modules as the order of parameter initializations, which often depends on a stateful random number generator, is different. Check Reproducibility for more details.

Lazy modules can be serialized with a state dict like other modules. For example:

Lazy modules can load regular torch.nn.Parameter s (i.e. you can serialize/deserialize initialized LazyModules and they will remain initialized)

Note, however, that the loaded parameters will not be replaced when doing a “dry run” if they are initialized when the state is loaded. This prevents using initialized modules in different contexts.

Check if a module has parameters that are not initialized.

Initialize parameters according to the input batch properties.

This adds an interface to isolate parameter initialization from the forward pass when doing parameter shape inference.
